

Learning Objectives

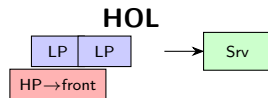
By the end of this lecture you will be able to:

- Distinguish head-of-line (HOL) priority from preemptive priority.
- Use SimPy's `PriorityResource` with explicit priority integers.
- Model preemption using `PreemptiveResource`.
- State the mean wait formulas for two-class M/M/1 priority queues.
- Analyse the trade-off: improving high-priority wait degrades low-priority wait.

Priority Disciplines: HOL and Preemptive

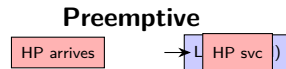
Head-of-line (HOL / non-preemptive)

High-priority customer goes to the front of the queue, but *does not interrupt* a customer already in service.



Preemptive

High-priority customer arriving to find a lower-priority customer in service *interrupts* it. The interrupted service resumes later (preemptive-resume) or restarts (preemptive-repeat).



SimPy PriorityResource: Lower Number = Higher Priority

```
import simpy, numpy as np
```

```
server = simpy.PriorityResource(env, capacity=1)
```

```
def customer(env, server, priority, mu, rng, waits):
```

```
    arrive = env.now
```

```
    # priority=0 jumps ahead of priority=1
```

```
    with server.request(priority=priority) as req:
```

```
        yield req
```

```
        waits[priority].append(env.now - arrive)
```

```
        yield env.timeout(rng.exponential(1/mu))
```

```
# High-priority (P1): priority=0
```

```
# Low-priority (P2): priority=1
```

```
env.process(customer(env, server, priority=0, mu=1, rng=rng, waits=waits))
```

```
env.process(customer(env, server, priority=1, mu=1, rng=rng, waits=waits))
```

- PriorityResource uses a heap ordered by priority value.
- Ties in priority are broken by arrival order (FCFS within class).

PreemptiveResource for True Preemption

```
server = simpy.PreemptiveResource(env, capacity=1)

def customer(env, server, priority, mu, rng, waits):
    arrive = env.now
    with server.request(priority=priority, preempt=True) as req:
        try:
            yield req
            waits[priority].append(env.now - arrive)
            yield env.timeout(rng.exponential(1/mu))
        except simpy.Interrupt as interrupt:
            # We were preempted; remaining service time
            remaining = interrupt.cause.usage_since
            # Re-queue with same priority to resume
            ...
```

Preemptive-resume

Store remaining service time and re-request with the same priority. On the next grant, yield `env.timeout(remaining)`.

Two-Class M/M/1: Mean Wait Formulas

Two Poisson classes: λ_1 (high), λ_2 (low), common μ . Let $\rho_i = \lambda_i/\mu$, $\rho = \rho_1 + \rho_2 < 1$.

HOL priority wait formulas

$$W_{q,1} = \frac{\rho/\mu}{1 - \rho_1}, \quad W_{q,2} = \frac{\rho/\mu}{(1 - \rho_1)(1 - \rho)}$$

Worked example: $\lambda_1 = 0.4$, $\lambda_2 = 0.4$, $\mu = 1$ ($\rho = 0.8$):

$$W_{q,1} = \frac{0.8}{1 - 0.4} = 1.33 \text{ min}$$

$$W_{q,2} = \frac{0.8}{(1 - 0.4)(1 - 0.8)} = 6.67 \text{ min}$$

$$W_q^{M/M/1} = \frac{0.8}{1 - 0.8} = 4.00 \text{ min (no priority)}$$

Priority helps class 1 by 67% but hurts class 2 by 67%.

When Priority Helps and When It Hurts

Priority benefits

- Reduces wait for time-critical customers (emergencies, SLA customers).
- Improves overall weighted cost if value of time differs across classes.
- No additional resource cost — purely a scheduling decision.

Priority costs

- Low-priority customers pay the full penalty: $W_{q,2}$ can be several times the unprioritised wait.
- Under heavy load, low-priority queue can grow unboundedly even when $\rho < 1$.
- Preemption adds implementation complexity and can cause service restarts.

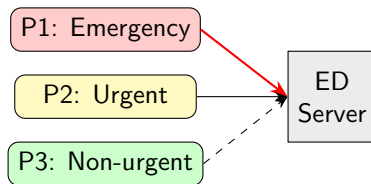
Key result: priority preserves the system-average wait ($L = \lambda W$ is unchanged); it redistributes wait *between* classes.

Application: Emergency Department Triage

- **P1 (Emergency):** chest pain, stroke — immediate service required.
- **P2 (Urgent):** fractures, lacerations — can wait < 30 min.
- **P3 (Non-urgent):** minor illness — wait up to 2 hours acceptable.

SimPy model: PriorityResource with priority values 0, 1, 2.

Service rate μ may also differ by class (P1 cases are complex).



Key Takeaways

Core ideas from L16

1. **HOL priority**: high-priority jumps the queue; does not interrupt service in progress.
2. **Preemptive priority**: high-priority interrupts ongoing service — use `PreemptiveResource`.
3. `SimPy PriorityResource`: `server.request(priority=p)` where *lower* p = higher priority.
4. Priority **redistributes** wait between classes but does not change system-total L or W .
5. The two-class formula shows class 2 pays a steep price: $W_{q,2}$ grows rapidly as ρ_1 increases.

Next: L17 — Non-stationary arrivals and the Lewis-Shedler thinning algorithm.